1.

a. Person (HkID, name)

Friendship(FriendA_HkID, FriendB_HkID, start_year, end_year)

Fk: FriendA_HkID → Person(HkID)

Fk: FriendB_HkID → Person(HkID)

b. Yes. Steps are as follow: step1: find "my" friends as group A
step2: find "my friend"s friends as group B
step3: check whether there exists the same person in group A and B

This need to self-join Friendship chart

c. First: It's hard to realize the "symetric concept" mentioned in the question in database level
Because when a new (PersonA, PersonB) is inserted, the symmetric one (PersonB, PersonA) will not be automatically created.

Second: The model is too simple to represent a friendship.
For example: Friendship have more conditions than just "start" and "end" such as "pause" "recover" etc.
At the same time, friendship is a process so it's hard to determine the precise start and end time in reality.

2.

a. HotelGroup(License_ID)

Hotel(License_ID, Hotel_ID)

Fk: License_ID → HotelGroup(License_ID)

HotelRoom(License_ID, Hotel_ID, Room_No)

Fk: License_ID, Hotel_ID → Hotel(License_ID, Hotel_ID)

b. No. Here, Hotel ID is a partial key, which means it's identical in a Hotel Group but not guaranteed in the whole world. If each Hotel ID is really unique all over the world, then the scenario can relatively recognized as the same [actually changed, but the function is still okay]. But, as I know, there are many "Regent" hotels in Hk. So, no matter from definition or reality aspect, the scenario will be changed.

3.

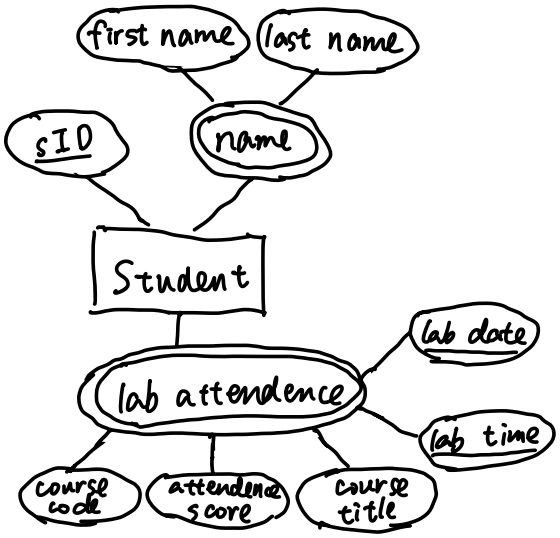
a. Can not achieve. Reasons are as below: this E/R model retains course information as attributes of "Attends" so there's a very simple problem that some courses don't take attendance and under this circumstance, these courses' info can't be retained. What's more, even asuming attendance is a must, there are still problems in updating, inserting, deleting since course info depends on "Attand". So, I think a independent "course" entity is needed.

b. Student(sID)
StudentName(sID, first_name, last_name)
Fk: sID → Student(sID)
LabAttendance(sID, lab_date, lab_time, course_code, attendance_score, course_title)
Fk: sID → Student(sID)

c. Student(sID)
StudentFirstName(sID, first_name)
Fk: sID → Student(sID)
StudentLastName(sID, last_name)
Fk: sID → Student(sID)

d. In StudentA, "name" is a multi-valued attribute, which means a student can have many names
In StudentB, "name" is a composite attribute, which means a student can only have one name
StudentB can be problematic. Example: since "first_name" and "last_name" are both multi-valued attribute, the name of a student will be in chaos because we don't know how to match them.
eg: first_name: A, B last_name: C, D, then a student's name can be AC, AD, BC, BD

e.




f. Student(sID)
StudentName(sID, first_name, last_name)
Fk: sID → Student(sID)
LabAttendance(sID, lab_date, lab_time, course_code, attendance_score, course_title)
Fk: sID → Student(sID)

here, although a student can only attend one lab in same data and time in reality, but there's no refinement in the new E/R model, so I think "course code" is needed to uniquely mark an attendance

g. Logical difference: in q3(b), LabAttendance is weak entity and can be "translated" that one student can only have one attendance in same date and time
in q3(f), LabAttendance is a multi-valued attribute of Student and allows one student to have more than one attendance in same date and time, which is impossible in reality.

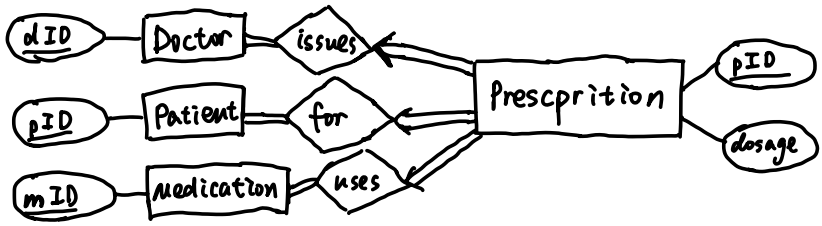
As mentioned above, the model in q3(b) can better express the relation between one Student and his/her attendances

4.
a. True. Since as shown that for each combination of D.p.M, the dosage is unique, so no inflence.
However, the new key allows the same D.p.M with different dosage, so the original and new key are actually different

b. Authorizes(D.M): (D1, M1), (D1, M2), (D2, M2)
Takes(P, M): (P1, M1), (P2, M2), (P1, M2)

c. issue: the tenary infos will be lost. eg: There is (D1, P1) in Treats, (D1, M2) in Authorizes and (P1, M2) in Takes, this info above can derive that (D1, P1, M2), which don't actually exist in reality.

d.




here, I think every prescription must have and only have one D. one P and one M according to the prescription info provided. However, as for the contrast aspect: "do each D. P. M must have at least 1 Prescription?", I'm not sure. According to the prescription info, all the D.P.M i.e. D1, D2, P1, P2, M1, M2 have all appeared, so it's reasonable to add a "at least one" refinement, a.k.a "=". But as far as I know, D will not always give prescriptions to P and we can use some kinds of M without prescription in reality. So I choose to add the refinement "=" based on question's info but in reality it should be "—"